UNIVERSIDADE FEEVALE
CURSO DE CIÊNCIA DA COMPUTAÇÃO
SISTEMAS OPERACIONAIS

Daniel Bier
Gabriel Fernando
Nícolas Benini
Samuel Horn
Sandro Dinarte
Yuri Campos

SIMULADOR DE ESCALONAMENTO DE PROCESSOS
ROUND ROBIN COM FEEDBACK

Relatório do Trabalho Final

Professora: Blanda Helena de Mello

Novo Hamburgo
2026

Sumário

1 Introdução

2 Descrição do algoritmo

3 Premissas

4 Estruturas de dados

- 4.1 PCB (Process Control Block)
- 4.2 Filas

5 Fluxo de execução

- 5.1 Criação
- 5.2 Seleção e execução
- 5.3 Preempção e feedback
- 5.4 Bloqueio e retorno de I/O
- 5.5 Finalização

6 Execução do projeto

- 6.1 Execução tradicional (Java)
- 6.2 Execução com Docker (bônus)

7 Saída do simulador

- 7.1 Log da simulação
- 7.2 Interpretação
- 7.3 Resumo final e métricas

8 Conclusão

1 Introdução

Um sistema operacional precisa decidir, a todo momento, qual dos processos prontos vai usar a CPU. Essa decisão é tomada pelo **escalonador**, e a forma como ele faz essa escolha afeta diretamente o tempo de resposta dos programas, o aproveitamento do processador e a justiça entre os processos.

O objetivo deste trabalho foi desenvolver um **simulador de escalonamento** que reproduz, de forma simplificada, como um sistema operacional gerencia processos prontos, em execução, bloqueados por entrada/saída (I/O) e finalizados. A estratégia implementada é o **Round Robin com Feedback**, com duas filas de prioridade e uma fila de I/O.

O simulador não executa programas reais: ele cria processos fictícios, cada um com seu PCB, e avança uma linha do tempo discreta, registrando cada evento relevante (criação, execução, preempção, pedido e retorno de I/O, finalização). Ao final, apresenta métricas que permitem avaliar o comportamento do algoritmo. O projeto foi implementado em **Java**, seguindo a orientação da disciplina de não modelar o simulador com orientação a objetos complexa.

2 Descrição do algoritmo

No **Round Robin** clássico, os processos prontos ficam numa fila e cada um recebe a CPU por, no máximo, uma fatia fixa de tempo chamada **quantum**. Se o processo não terminar dentro do quantum, ele sofre **preempção**: o sistema o retira da CPU e o coloca de volta na fila. Isso impede que um processo longo monopolize o processador.

A variação **com Feedback** acrescenta mais de uma fila, com prioridades diferentes, e faz os processos **mudarem de fila conforme o seu comportamento**. No nosso simulador:

- a fila **ALTA** é sempre atendida primeiro; a fila **BAIXA** só é usada quando a ALTA está vazia;
- dentro de cada fila, a ordem é FIFO (o primeiro que entrou é o primeiro a sair);
- um processo que **usa o quantum inteiro sem terminar** é rebaixado para a fila BAIXA;
- um processo que faz I/O volta para uma fila que depende do dispositivo usado.

A ideia por trás do feedback é favorecer processos curtos e interativos. Processos que consomem muita CPU (*CPU-bound*) vão perdendo prioridade, enquanto processos que liberam a CPU logo para fazer I/O voltam rapidamente a ser atendidos. As regras de entrada nas filas, definidas no enunciado do trabalho, estão resumidas na Tabela 1.

Situação	Fila de destino
Processo novo	ALTA
Usou o quantum inteiro e não terminou (preempção)	BAIXA
Retorno de I/O de **disco**	BAIXA
Retorno de I/O de **fita magnética**	ALTA
Retorno de I/O de **impressora**	ALTA

Tabela 1 — Regras de feedback entre as filas.

3 Premissas

Antes de implementar, o grupo definiu as premissas da simulação. A principal decisão foi usar um **cenário fixo**, em vez de gerar processos aleatoriamente. Com valores fixos, a simulação produz sempre a mesma saída, o que facilita conferir o resultado à mão, comparar execuções e explicar cada evento na apresentação. Por isso, a semente aleatória não se aplica.

Premissa	Valor adotado
Algoritmo	Round Robin com Feedback
Quantum	3 unidades de tempo
Quantidade máxima de processos	5
Critério de criação	Todos criados no instante t=0, filhos do processo inicial (PPID = 0)
Tempo de CPU por processo	P1 = 6, P2 = 5, P3 = 4, P4 = 3, P5 = 2 unidades (faixa de 2 a 6)
Duração do I/O	Disco = 4, fita = 3, impressora = 5 unidades (faixa de 3 a 5)
Momento do I/O	P2 pede disco após 2 unidades de CPU; P3 pede fita após 1; P4 pede impressora após 1; P1 e P5 não fazem I/O
Quantidade de I/O	No máximo um pedido de I/O por processo
Fila de I/O	Uma única fila compartilhada pelos três dispositivos; cada pedido é atendido em paralelo, sem disputa pelo mesmo dispositivo
Critério de finalização	A simulação termina quando todos os processos estão FINALIZADOS
Semente aleatória	Não se aplica (valores fixos, simulação reprodutível)

Tabela 2 — Premissas da simulação.

Os processos foram escolhidos para exercitar todas as regras do algoritmo: P1 é longo e sem I/O (sofre preempção), P5 é curto e sem I/O (termina no primeiro quantum), e P2, P3 e P4 usam, cada um, um dos três dispositivos, cobrindo os dois destinos possíveis de retorno.

4 Estruturas de dados

A orientação da disciplina para Java é não modelar o simulador com classes de domínio e objetos complexos, e sim usar uma classe principal com métodos estáticos e registros simples. Seguimos essa linha: toda a lógica está na classe `Escalonador`, no método `main` e em um método estático auxiliar (`atualizarIo`). As classes internas existem apenas para agrupar dados, funcionando como uma `struct` da linguagem C.

4.1 PCB (Process Control Block)

Cada processo é representado por um registro `Processo`, que guarda os campos exigidos (PID, PPID, prioridade e status) e as informações de controle necessárias para a simulação:

```
static class Processo {
    int pid, ppid, tempoRestante, tempoTotal, cpuUsada,
        tipoIo, tempoIo, momentoIo, prioridade, fim;
    String status;
    boolean ioFeito;
    ...
}
```

Campo	Finalidade
`pid` / `ppid`	Identificador do processo e do processo pai (0 = processo inicial).
`status`	Estado atual: NOVO, PRONTO, EXECUTANDO, BLOQUEADO ou FINALIZADO.
`prioridade`	Fila em que o processo deve entrar: 1 = ALTA, 2 = BAIXA.
`tempoTotal`	Tempo total de CPU de que o processo precisa.
`tempoRestante`	Quanto de CPU ainda falta; ao chegar a zero, o processo é finalizado.
`cpuUsada`	Quanto de CPU o processo já consumiu; usado para saber quando ele pede I/O.
`tipoIo`	Dispositivo do próximo I/O: 0 = nenhum, 1 = disco, 2 = fita, 3 = impressora.
`tempoIo` / `momentoIo`	Duração do I/O e após quantas unidades de CPU ele é solicitado.
`ioFeito`	Indica se o I/O já aconteceu (cada processo faz no máximo um).
`fim`	Instante de término, usado no cálculo das métricas.

Tabela 3 — Campos do PCB.

O `status` é atualizado a cada transição de estado: o processo nasce NOVO, passa a PRONTO ao entrar numa fila, fica EXECUTANDO enquanto ocupa a CPU, BLOQUEADO enquanto espera o I/O, e FINALIZADO quando seu tempo de CPU acaba.

4.2 Filas

As três filas exigidas foram implementadas com `ArrayList`, usadas em ordem FIFO (insere com `add` no fim e retira com `remove(0)` do início):

- `alta` — processos prontos de prioridade alta;
- `baixa` — processos prontos de prioridade baixa;
- `ioLista` — processos bloqueados. Cada elemento é um registro `IoItem`, que guarda o processo e o campo `espera`, isto é, quantas unidades de tempo ainda faltam para o I/O terminar.

Optamos por uma **única fila de I/O** para os três dispositivos, como permite o enunciado. O tipo do dispositivo fica no PCB e é consultado no retorno, para decidir a fila de destino. Filas separadas por dispositivo ficaram como melhoria futura.

5 Fluxo de execução

O simulador funciona como uma linha do tempo controlada pelo laço principal (`while`), que se repete até todos os processos terminarem. Cada volta do laço segue quatro passos: escolher o processo, decidir por quanto tempo ele executa, executá-lo e decidir para onde ele vai ao sair da CPU.

5.1 Criação

No início, os cinco processos são criados com status NOVO, prioridade ALTA e PPID 0, e inseridos na fila ALTA, onde passam ao estado PRONTO. Cada criação é registrada no log no instante t=0.

5.2 Seleção e execução

A seleção dá preferência à fila ALTA; a BAIXA só é consultada se a ALTA estiver vazia. Se as duas estiverem vazias, a CPU fica ociosa por uma unidade de tempo, o que é registrado no log e contabilizado nas métricas.

Escolhido o processo, calcula-se a fatia de execução: no máximo um quantum, ou menos se faltar pouco para o processo terminar (`Math.min(quantum, tempoRestante)`). Se o processo tiver um I/O pendente que cai antes do fim dessa fatia, ele executa apenas até o momento de pedir o I/O.

Um ponto importante da implementação é que o relógio avança **de uma em uma unidade** durante a execução, e a cada unidade a fila de I/O é atualizada pelo método `atualizarIo`. Assim, os dispositivos trabalham em paralelo com a CPU: enquanto um processo executa, os I/O dos outros continuam andando, e um retorno é registrado no instante exato em que acontece.

5.3 Preempção e feedback

Se, ao fim da fatia, o processo não terminou nem pediu I/O, significa que ele usou o quantum inteiro. Ele sofre preempção: volta ao estado PRONTO, recebe prioridade BAIXA e vai para o fim da fila BAIXA. Esse rebaixamento é o feedback do algoritmo. Um processo que já está na fila BAIXA e sofre nova preempção permanece nela, já que existem apenas dois níveis.

5.4 Bloqueio e retorno de I/O

Quando o processo atinge o momento do seu I/O, ele sai da CPU com status BLOQUEADO e entra na `ioLista` com o tempo de espera igual à duração do dispositivo. A cada unidade de tempo, a espera de todos os itens é decrementada. Quando chega a zero, o processo volta a PRONTO e é inserido na fila definida pelo dispositivo: BAIXA para disco; ALTA para fita e impressora.

5.5 Finalização

Quando o tempo restante de CPU chega a zero, o processo recebe status FINALIZADO e seu instante de término é guardado. Se o fim do processo coincide com o fim do quantum, ele é finalizado, e não preemptado, porque essa verificação vem antes. Quando todos os processos terminam, o laço se encerra e o simulador imprime o resumo final.

6 Execução do projeto

6.1 Execução tradicional (Java)

É necessário ter o JDK instalado (o projeto foi testado com Java 21). A partir da raiz do repositório:

```
javac -d out src/Escalonador.java
java -cp out Escalonador
```

O primeiro comando compila o código e grava os arquivos `.class` na pasta `out/`, mantendo a pasta `src/` só com o código-fonte. O segundo executa a classe `Escalonador`, indicando com `-cp` (*classpath*) onde encontrá-la. Não é necessário nenhum arquivo de entrada: o cenário está definido no próprio código.

6.2 Execução com Docker (bônus)

O repositório inclui um `Dockerfile` baseado no modelo da disciplina para Java:

```
FROM eclipse-temurin:21-jdk
WORKDIR /app
COPY src/ ./src/
RUN javac src/*.java -d out
CMD ["java", "-cp", "out", "Escalonador"]
```

A imagem parte do JDK 21 oficial (Eclipse Temurin), copia o código-fonte, compila durante o *build* e, ao executar o container, roda o simulador mostrando a saída no terminal. O arquivo `.dockerignore` evita copiar para a imagem o histórico do Git e arquivos compilados localmente. Os comandos são:

```
docker build -t so-escalonador-grupo-x .
docker run --rm so-escalonador-grupo-x
```

Com isso, o simulador pode ser executado em qualquer máquina com Docker, sem instalar o Java manualmente e sempre com a mesma versão do compilador.

7 Saída do simulador

7.1 Log da simulação

Cada linha do log começa com o instante em que o evento ocorreu (`[t=XXX]`). A execução completa do cenário produz:

```
[config] quantum=3 | processos=5 | valores fixos
[t=000] P1 criado -> fila ALTA
[t=000] P2 criado -> fila ALTA
[t=000] P3 criado -> fila ALTA
[t=000] P4 criado -> fila ALTA
[t=000] P5 criado -> fila ALTA
[t=000] CPU executa P1 por 3 unidades
[t=003] P1 sofreu preempcao -> fila BAIXA
[t=003] CPU executa P2 por 2 unidades
[t=005] P2 solicitou I/O DISCO por 4 unidades
[t=005] CPU executa P3 por 1 unidades
[t=006] P3 solicitou I/O FITA por 3 unidades
[t=006] CPU executa P4 por 1 unidades
[t=007] P4 solicitou I/O IMPRESSORA por 5 unidades
[t=007] CPU executa P5 por 2 unidades
[t=009] P2 retornou do DISCO -> fila BAIXA
[t=009] P3 retornou da FITA -> fila ALTA
[t=009] P5 finalizado
[t=009] CPU executa P3 por 3 unidades
[t=012] P4 retornou da IMPRESSORA -> fila ALTA
[t=012] P3 finalizado
[t=012] CPU executa P4 por 2 unidades
[t=014] P4 finalizado
[t=014] CPU executa P1 por 3 unidades
[t=017] P1 finalizado
[t=017] CPU executa P2 por 3 unidades
[t=020] P2 finalizado
[fim] Processos finalizados: 5/5 | tempo total: 20
```

A Figura 1 mostra a mesma execução como um diagrama de tempo: a primeira linha indica qual processo ocupa a CPU em cada unidade, e as demais mostram os períodos em que cada processo esteve bloqueado em I/O.

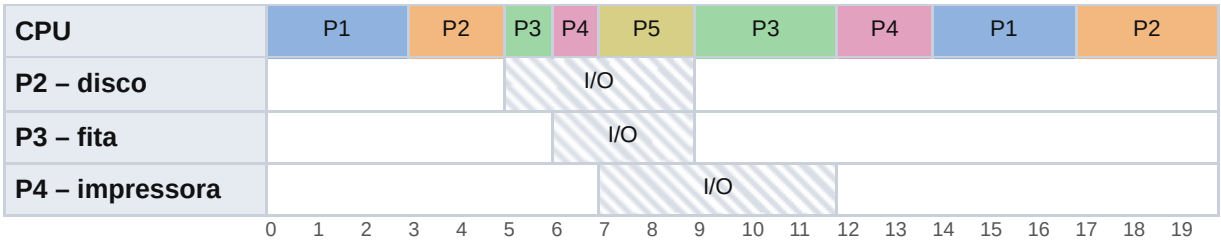


Figura 1 — Diagrama de tempo da execução (cada coluna é uma unidade de tempo).

7.2 Interpretação

- **t=0 a 3 — preempção.** P1 é o primeiro da fila ALTA e precisa de 6 unidades. Ele executa um quantum inteiro (3) sem terminar, sofre preempção e é rebaixado para a fila BAIXA.
- **t=3 a 7 — bloqueios por I/O.** P2, P3 e P4 executam apenas até o momento de pedir I/O (2, 1 e 1 unidades) e são bloqueados. Eles liberam a CPU antes do fim do quantum, que é o comportamento típico de processos com bastante I/O.
- **t=7 a 9 — finalização dentro do quantum.** P5 precisa de apenas 2 unidades e termina sem sofrer preempção.
- **t=9 — retornos de I/O e feedback por dispositivo.** Durante a execução de P5, o disco de P2 e a fita de P3 terminam. P2 volta para a fila BAIXA (regra do disco) e P3 volta para a fila ALTA (regra da fita).
- **t=9 a 14 — prioridade da fila ALTA.** Mesmo com P1 e P2 esperando na fila BAIXA, a CPU atende P3 e depois P4, que voltou da impressora em t=12 para a fila ALTA. Isso mostra a preferência da fila ALTA e o paralelismo entre CPU e I/O: a impressora de P4 terminou enquanto P3 executava.
- **t=14 a 20 — fila BAIXA.** Só quando a fila ALTA esvazia é que P1 e P2 são atendidos, na ordem em que entraram na fila BAIXA. Os dois terminam dentro do quantum.

A CPU não ficou ociosa em nenhum momento: sempre havia pelo menos um processo pronto enquanto outros estavam em I/O.

7.3 Resumo final e métricas

Ao final, o simulador imprime o PCB de cada processo e as métricas da execução:

```
PID | PPID | CPU | I/O        | Fim | Turnaround | Espera | Status
P1  |    0 |   6 | NENHUM     |  17 |         17 |     11 | FINALIZADO
P2  |    0 |   5 | DISCO      |  20 |         20 |     11 | FINALIZADO
P3  |    0 |   4 | FITA       |  12 |         12 |      5 | FINALIZADO
P4  |    0 |   3 | IMPRESSORA |  14 |         14 |      6 | FINALIZADO
P5  |    0 |   2 | NENHUM     |   9 |          9 |      7 | FINALIZADO
Preempcoes: 1
I/O -> DISCO=1 | FITA=1 | IMPRESSORA=1
Espera media: 8.00 | Turnaround medio: 14.40
CPU ociosa: 0 unidades (0.0%)
```

Como as métricas são calculadas
Turnaround = instante de término − instante de chegada. Como todos chegam em t=0, é o próprio instante de término.
Espera = turnaround − tempo de CPU − tempo em I/O, ou seja, o tempo em que o processo ficou parado nas filas de prontos.
Exemplo, P2: 20 − 5 (CPU) − 4 (disco) = 11 unidades de espera.

As métricas confirmam o efeito do feedback. Os processos que fizeram I/O de fita e impressora (P3 e P4) tiveram as menores esperas (5 e 6), porque voltaram para a fila ALTA. Já P1, rebaixado por preempção, e P2, que voltou do disco para a fila BAIXA, tiveram a maior espera (11) e foram os últimos a terminar. O tempo total foi de 20 unidades, igual à soma dos tempos de CPU (6 + 5 + 4 + 3 + 2), o que confirma que o processador não ficou parado.

8 Conclusão

O simulador cumpre os requisitos do trabalho: executa processos com Round Robin, aplica o feedback entre as filas de alta e baixa prioridade, trata os três dispositivos de I/O com o retorno para a fila correta, mantém um PCB com PID, PPID, prioridade e status, e registra cada evento numa linha do tempo, com métricas ao final.

Aprendizados. Implementar o escalonador deixou concretos conceitos que na teoria parecem simples. Ficou claro como a preempção garante que nenhum processo monopolize a CPU, como o feedback favorece processos curtos e com I/O, e como a CPU e os dispositivos trabalham em paralelo: enquanto um processo executa, outros estão bloqueados esperando o I/O terminar.

Dificuldades. A principal dificuldade foi justamente a sincronização entre a CPU e o I/O. Na primeira versão, o relógio avançava um quantum inteiro por vez, mas a fila de I/O era atualizada apenas uma vez a cada volta do laço. Com isso, o I/O "andava mais devagar" que o relógio: P4 pedia a impressora em t=7, por 5 unidades, e só voltava em t=18, em vez de t=12. A correção foi fazer o tempo avançar de uma em uma unidade, atualizando a fila de I/O a cada passo. Outra decisão de projeto foi interpretar a restrição de "sem orientação a objetos" em Java, o que resolvemos usando registros simples e lógica em métodos estáticos.

Melhorias futuras.

- Gerar processos e tempos aleatoriamente, com semente fixa para manter a reprodutibilidade, ou ler o cenário de um arquivo em `input/`;
- Permitir chegadas em instantes diferentes, mais de um I/O por processo e quantum configurável, comparando o resultado com o Round Robin simples;
- Implementar uma fila separada por dispositivo, com disputa quando dois processos pedem o mesmo dispositivo.